HOCHSCHULE ESSLINGEN

BETRIEBSSYSTEME (BS)

II. BASH PROGRAMMIERUNG

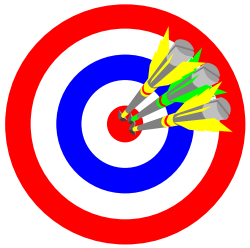
PROF. DR.-ING. RAINER KELLER
PROF. DR. DENNIS GREWE

AGENDA

- Was ist die Bash?
- Wie gebe ich Befehle in die Bash ein?
- Wie navigiere ich im Dateisystem mit der Bash?
- Wie entwickle ich Programme für die Bash?

 © Copyright Material. All rights reserved.

ZIELE



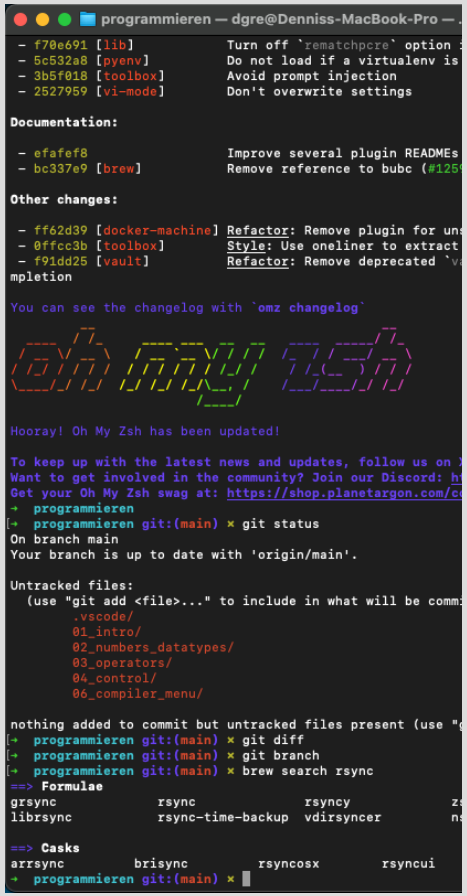
- Entwicklung von kleinen Programmen für die Bash erlernen
- Wichtigste Kommandos in der Bash zur Navigation und Interaktion mit dem Dateisystem.

- HINWEIS: Machen Sie sich Notizen in den jeweiligen Grafiken

© Copyright Material. All rights reserved.

HOCHSCHULE ESSLINGEN

EINFÜHRUNG IN DIE BASH



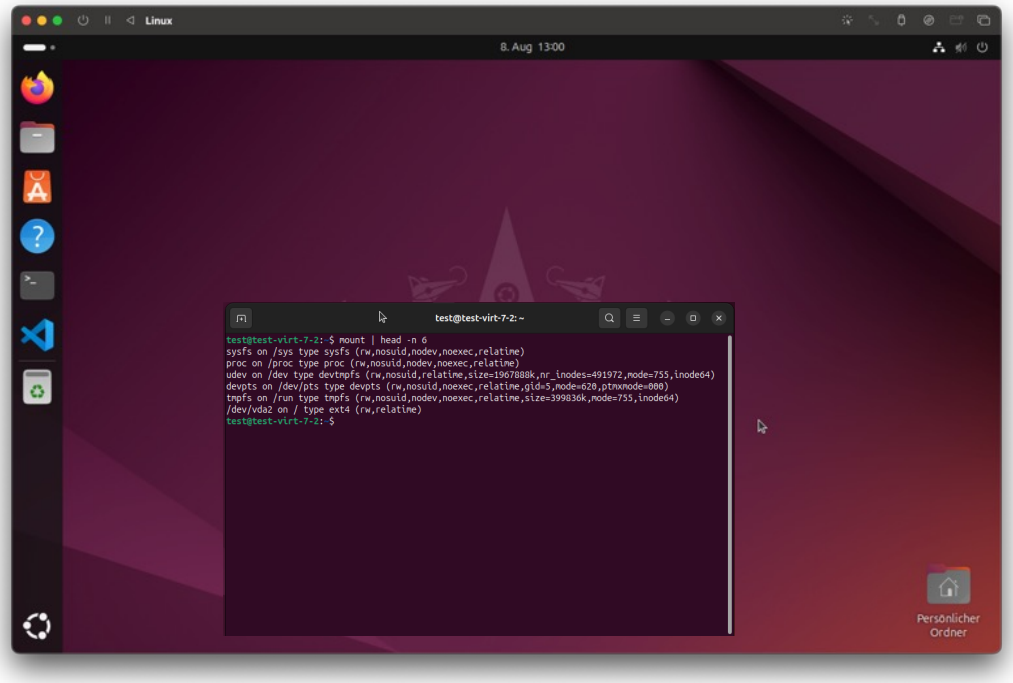
Quelle: Dennis Grewe

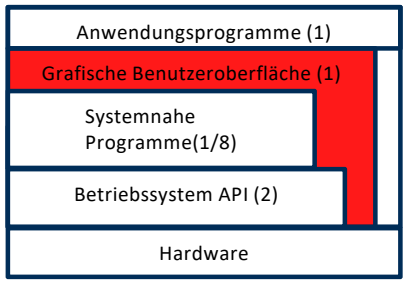
HINWEIS

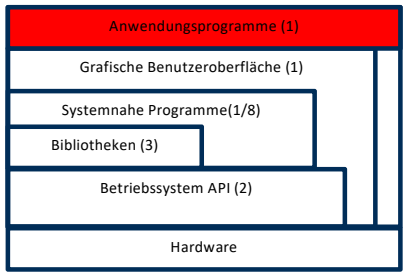
- Die nachfolgenden Inhalte sollten Sie direkt in Ihrer Linux-VM oder an einem Linux-Rechner machen

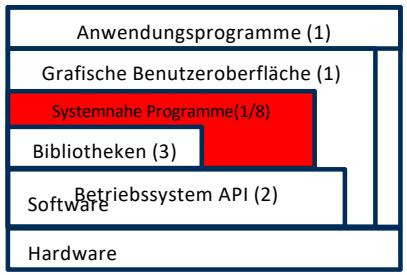
 © Copyright Material. All rights reserved.

GRAFISCHE OBERFLÄCHE (GUI) VS. KOMMANDOZEILE (SHELL)





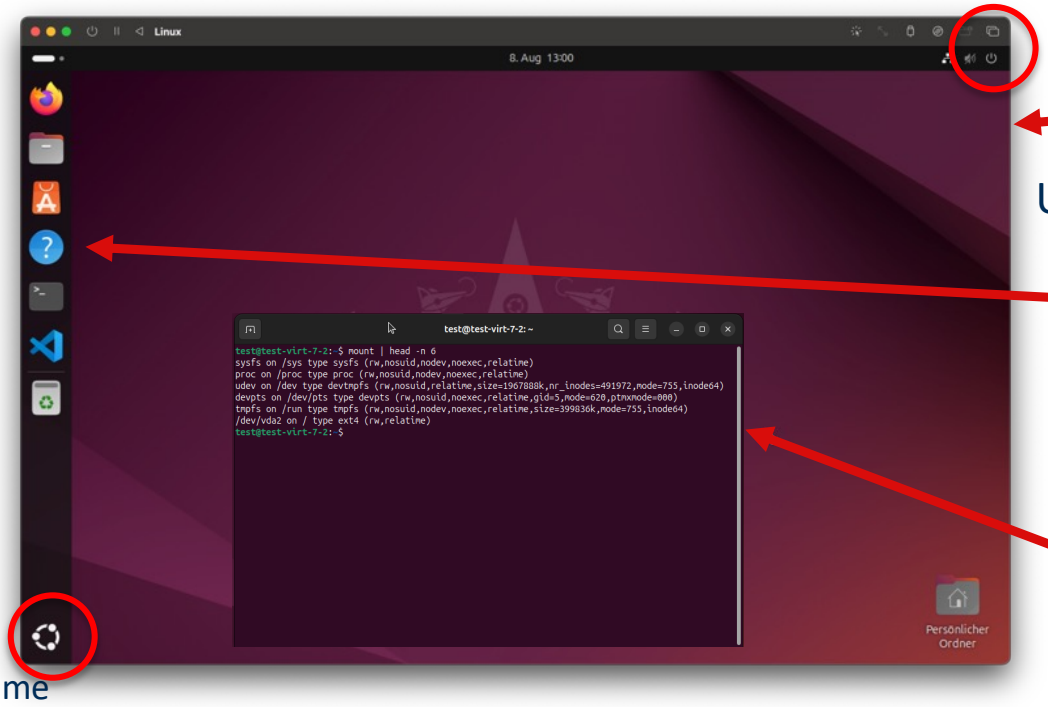




- Grafische Programme erlauben eine intuitive und effiziente Arbeitsweise.
- Systemdetails und systemnahe Informationen bekommt man aber meist nur über Programme die von der **Shell** gestartet werden, indem man über die Kommandozeile (**Terminal**) interagiert.

© Copyright Material. All rights reserved.

GRAFISCHE OBERFLÄCHE (GUI) VS. KOMMANDOZEILE (SHELL)



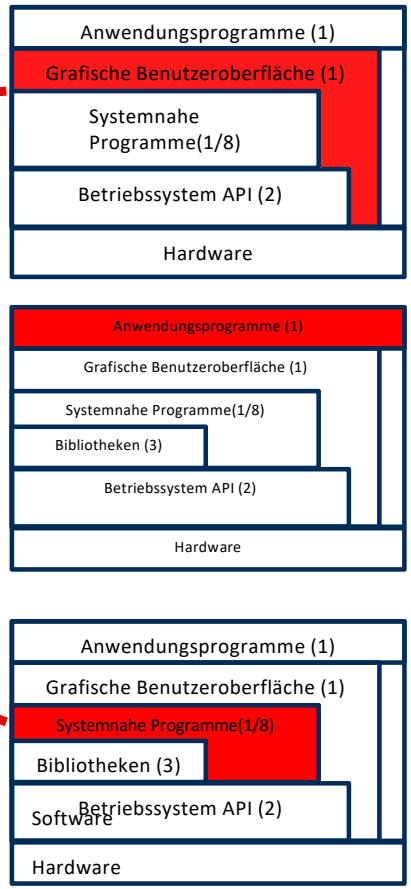
Ausschalten

Ubuntu Desktop

Anwendungen

Terminal

Weitere Programme

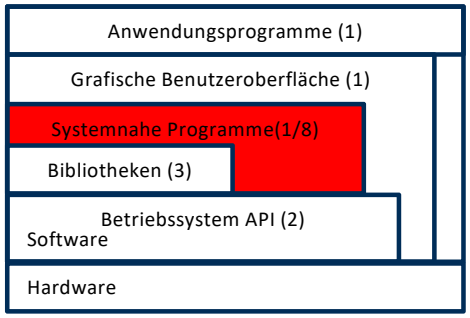


- Grafische Programme erlauben eine intuitive und effiziente Arbeitsweise.
- Systemdetails und systemnahe Informationen bekommt man aber meist nur über Programme die von der **Shell** gestartet werden, indem man über die Kommandozeile (**Terminal**) interagiert.

© Copyright Material. All rights reserved.

UNIX SHELL (WIEDERHOLUNG)

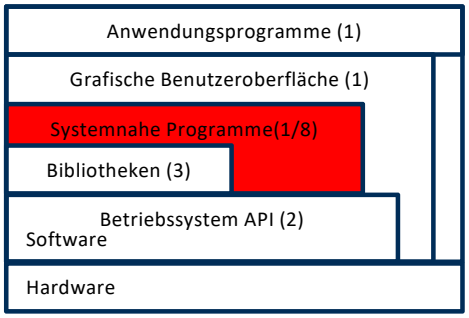
- Eine **Shell** ist die Kommandozeile mit der man Programme startet
- Die Shell ist somit die niedrigste, systemnahe Software, mit der ein Benutzer (engl. *user*) interagiert.
- Shells haben Eigenschaften von Programmiersprachen. Es gibt folgende bekannte Shells:
  - Bourne Shell (Kommando: `sh`) --> Standard in Linux
  - C-Shell (`csh`)
  - Korn Shell (`ksh`)
  - Z-Shell (`zsh`) --> Standard in macOS
- Es sind nicht alle genannten Shells automatisch in Linux installiert. Auf jeden Fall ist die **`bash`** fester Bestandteil aller verbreiteter Linux Distributionen.



© Copyright Material. All rights reserved.

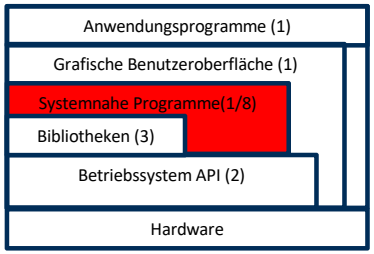
UNIX BASH (WIEDERHOLUNG)

- Mit der UNIX bash können wir:
  - Systemnahe Programme starten
  - Grafische Programme starten
  - Programme stoppen (engl. terminate / kill)
  - Komplexe Shell-Skripte erstellen und ausführen!
- In der Kommandozeile steht die Eingabeaufforderung:
  - `rakeller@James:~$|`
  - Hier können Sie Programme starten …
  - Programme pausieren (`STRG/CTRL+Z`) und beenden (`STRG/CTRL+C`)
- Zu allen Befehlen gibt es „Manual Pages“ (bspw. `man bash`) --> RTFM ☺



© Copyright Material. All rights reserved.

UNIX BASH: EINGABE SYNTAX



- Die Eingabestruktur (Syntax) sieht wie folgt aus (siehe Man-Pages):

```
$ program [Param1] [P2|P3] -p pid ↵
```

Alles was auf der SHELL einzugeben ist, steht nach `$`.

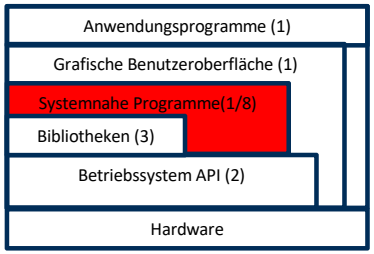
Optionale Parameter in eckigen Klammern

Schließen sich Parameter aus, werden sie mittels dem Oder | angegeben.

Argumente für die Parameter sind unterstrichen angegeben...

© Copyright Material. All rights reserved.

UNIX BASH: EINGABE SYNTAX



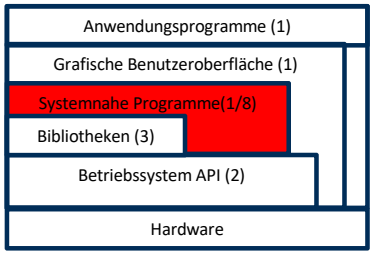
- Die Eingabestruktur (Syntax) sieht wie folgt aus (siehe Man-Pages):

```
$ program [Param1] [P2|P3] -p pid ↵
```

- UNIX tools verwenden meist ähnliche Parameter, die man sich gut merken kann wie:
  - `-i` Ignore case (Groß-/Kleinschreibung egal)
  - `-h` Human readable (Einfach lesbar)
  - `-r` Recursive (Alle Dateien & in Unterordnern)
  - `-p PID` Process ID wird als Argument angegeben.
  - `-l` Long format (z.B. ls zeigt alle Dateien)

 © Copyright Material. All rights reserved.

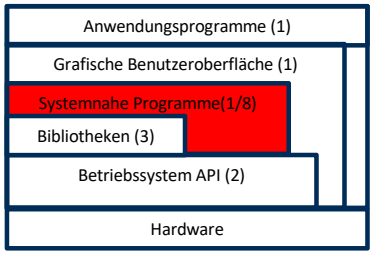
UNIX BASH: WEITERE EIGENSCHAFTEN



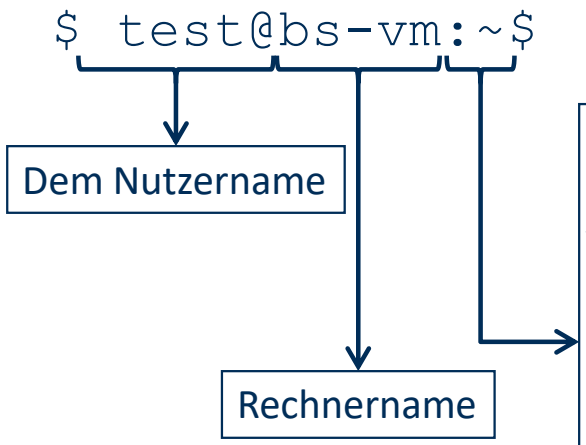
- `STRG+Z` / `CTRL+Z` pausiert den laufenden Prozess
  - Verbraucht somit keine weitere CPU Zeit mehr
- Der pausierte Prozess kann mit `bg` in den Hintergrund, bzw. mit `fg` wieder in den Vordergrund gebracht werden
- Mit & werden Programme sofort in im als Hintergrundprozess gestartet:
  - `rakeller@James:~$ firefox &↵`
    `[1] 8631`
- Programme können mit `STRG+C` / `CTRL+C` beendet (kill) werden
- Oder mittels Prozess-ID (PID) gestoppt werden:
  - `rakeller@James:~$ kill 8631 ↵ (stoppt Firefox Prozess!)`

 © Copyright Material. All rights reserved.

UNIX BASH: STRUKTUR KOMMANDOZEILE



- Die Kommandozeile setzt sich zusammen aus:

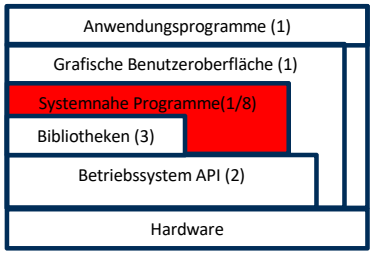


Dem Verzeichnis (Directory), in dem man sich befindet – hier das Heimat-Verzeichnis, auch gespeichert in der Umgebungs-variablen `$HOME`, oder auch kurz ~ (Tilde) genannt!
Mehr dazu in den folgenden Folien...

Das `$`-Zeichen zeigt an, daß dahinter Befehle folgen.
In unserm Foliensatz verwenden wir immer `$`, aber es kann auch andere Zeichen wie > gesetzt sein.
Hint: Siehe die Umgebungsvariable `PS1`.

© Copyright Material. All rights reserved.

UNIX BASH: STRUKTUR KOMMANDOZEILE



- Die Kommandozeile setzt sich zusammen aus:

```
$ dgrewe@bs-vm:~$|
```

- Die Kommandozeile wird definiert durch die Umgebungsvariable `PS1`:

```
$ echo $PS1↵        (Ausgabe der Variablen PS1)
$ export PS1='User \u on \h in directory \w : '↵
```

setzt die Variable `PS1` auf einen etwas zu ausführlichen Schreibstil:

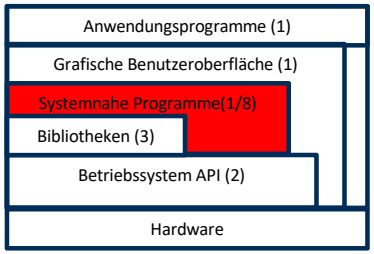
```
User dgrewe on bs-vm in directory ~ : |
```

- Die anderen Zeichen `@`, `:`, `$`, (Leerzeichen) sind nur syntaktischer Zucker
- Man kann mit `PS1` ganz verrückte Spielereien setzen (Farbe, Datum, Sub-shell, …)

[Link: PS1 Guide]

 © Copyright Material. All rights reserved.

UNIX BASH: DER BEFEHL `CD`



- Nach dem Start „zeigt“ die Shell auf das Heimverzeichnis des Benutzers `$HOME`, ~

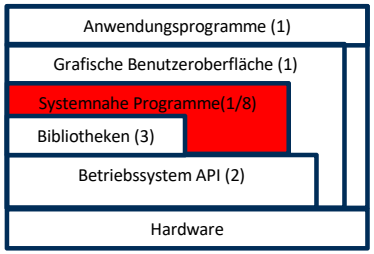
```
$ dgrewe@bs-vm:~$
```

- Mit dem Befehl `cd` (change directory) können wir uns frei in Verzeichnissen bewegen:

```
$ cd /home/↵    (Wechsel in absolutes Verzeichnis)
$ cd .. ↵       (Wechsel relativ: hier Verzeichnis hoch)
$ cd ↵          (shortcut für Wechsel nach $HOME)
```

 © Copyright Material. All rights reserved.

UNIX BASH: DER BEFEHL `LS`



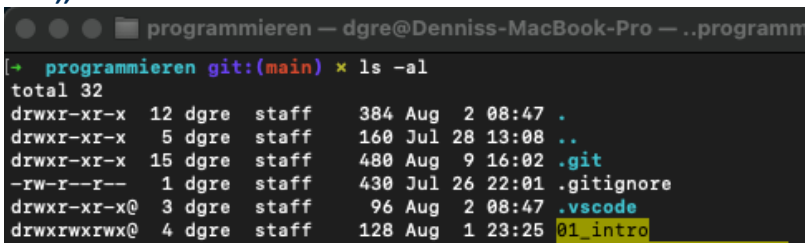
- Mit dem Befehl `ls` (list) können wir uns die Inhalte eines Verzeichnis ausgeben lassen:

```
$ dgrewe@bs-vm:~$ ls ↵
```

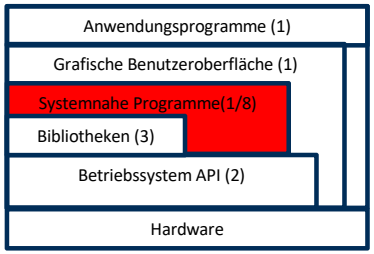
- Weitere Beispiele von ls:

```
$ ls -a ↵        (zeigt alle versteckten Inhalte)
$ ls -al ↵       (zeigt alle Dateien im Long-Format)
```

- Vor allem letzterer Befehl zeigt die Datei-Rechte oder auch „UNIX Permissions“ genannt



UNIX BASH: MAN PAGES

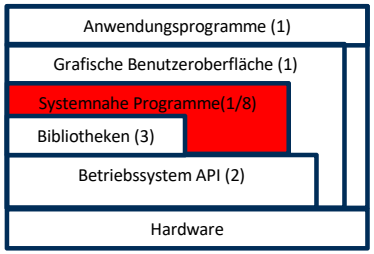


- Zu allen Kommandos finden Sie Hilfe in den „Manual Pages“ (man pages)
  - `$ man ↵  # für manual selbst`
  - `$ man bash ↵  # Hilfsseite zur Bash`
- Die „Man-Pages“ sind in Bereiche 1-8 gruppiert:
  - `$ man 1 bash ↵  # 1=System-& Benutzerprogramme`
  - `$ man 2 gettimeofday ↵  # 2=Systemcalls & Kernelparameter`
  - `$ man 3 printf ↵  # 3=Bibliotheksaufrufe`
  - `$ man 4 kmem ↵  # 4=Gerätetreiber & Netz.-prot.`
  - `$ man 5 ssh_config ↵  # 5=Versch. Konfigurationdateien`
  - `$ man 7 tcp ↵  # 7=Netzwerk`
  - `$ man 8 netstat ↵  # 8=Systemadmin Tools`

- Obige Nummern gelten für Linux (nicht macOS, da dies auf BSD basiert).
  Diese durchsuchen Inhalte in der `$MANPATH` Variable, der Default ist:
  `/usr/share/man/:/usr/local/share/man`

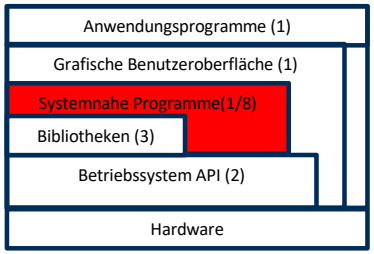
 © Copyright Material. All rights reserved.

UNIX BASH: DER `ECHO` BEFEHL



- Mit Hilfe des Befehls `echo` lassen sich Ausgaben auf der Kommandozeile schreiben
  - `$ echo "Hello World" ↵`
  - `Hello World  # gibt "Hello World" aus`

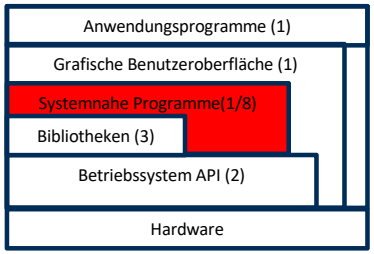
UNIX BASH: VARIABLEN `VAR`



- Mit Hilfe des Befehls var lassen sich Variablen setzen:
  - `$ var=4711 ↵`
  - `$ echo "var ist $var" ↵  # Ausgabe Inhalt var`
- Mit `var=…` wird ein Wert zugewiesen, mit `$var` wird der Wert gelesen. (Achtung, Groß- vs. Kleinschreibung, var ist eine andere Variable als Var...)
- Länger gültige Variablen werden `GROSS` geschrieben (erhöht Unterscheid- und Lesbarkeit)
- Mittels `{ }` kann das Arbeiten mit Variablen explizit gemacht werden und testet auf leere/nicht-existente Variablen -> weniger Fehler
  - `$ echo ${GROSS}`

 © Copyright Material. All rights reserved.

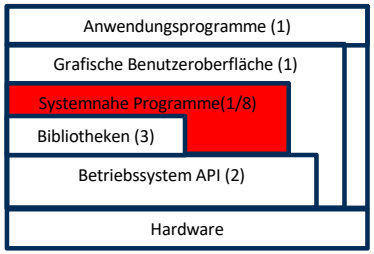
UNIX BASH: UMGEBUNGSVARIABLEN



- Umgebungsvariablen sind wie echte Variablen und werden an Programme übergeben. Variablen kann man mit C-Funktion `ptr=getenv("VARNAME")` lesen (siehe `$ man 3 getenv`)
- Es gibt eine Menge sinnvoller Umgebungsvariablen, z.B.:
  - `HOME  # zeigt Heimatverzeichnis des Nutzers`
  - `PWD  # aktuelles Arbeitsverzeichnis (present working directory)`
  - `OLDPWD # vorheriges Arbeitsverzeichnis`
  - `USER  # der Benutzername`
- Umgebungsvariablen setzt man durch Zuweisung und ließt sie mit `$VAR` aus
  - `$ export MEINE_VARIABLE=1`

 © Copyright Material. All rights reserved.

UNIX BASH: EINGABEN



- Die Kommandozeile ist wie ein Editor mit vielen Tastenkürzeln:
  - CTRL-A # Cursor springt an Zeilenanfang
  - CTRL-E # Cursor springt and Zeilenende
  - CTRL-XX # merkt sich Position und springt zur letzten zurück
  - ESC← # (Escape+Backspace) löscht ganzes Wort vor Cursor
  - ESC-D # Löscht Wort nach Curser und speichert in einem Puffer
  - CTRL-Y # Yank: fügt den Puffer an dieser Stelle wieder ein
  - CTRL-C # Bricht die derzeitige Zeile ab
  - CTRL-L # Clear Screen, wie "clear", leert das Fenster
  - CTRL-R # Rückwärtssuche bisheriger Befehle (mächtiger Befehl)
  - TAB # vervollständigt Filename (wie Autovervollständigung)

 © Copyright Material. All rights reserved.

UNIX BASH: BEFEHLE

- Weitere Bash Kommandos und Befehle sind in dem **Cheat Sheet Bash** aufgelistet
- **Bitte lernen Sie diese im Eigenstudium!**

[Bild von vecstock auf Freepik]

© Copyright Material. All rights reserved.

HOCHSCHULE ESSLINGEN

BESONDERE KOMMANDOS UND PROGRAMME DER BASH

BASH: PROGRAMMIERBARKEIT

HOCHSCHULE ESSLINGEN

- Die “Mächtigkeit“ aller Shells liegt in der Programmierbarkeit!
- UNIX verfolgt dabei das `KISS` Prinzip: **K**eep **I**t **S**imple, **S**tupid
  - **1 Programm macht 1 Sache** -> diese aber gut.
  - Beispiele: `sort, date, find, …`
- UNIX Programme lassen sich kombinieren:
  - Die “Standardausgabe“ eines Programms (File-Descriptor 1) lässt sich in die Eingabe des nächsten Programms weiterreichen (mittels Pipe “|“ Symbol)
    - `$ sort file.txt | tail -n 5 | grep -E 'd$'`
    - Ablauf: Sortiert `file.txt` alphanumerisch, nimmt die letzten `5` Zeichen und such mit regulärem Ausdruck nach Zeilen die auf “d“ enden!

 © Copyright Material. All rights reserved.

BASH: RÜCKGABEWERTE

HOCHSCHULE ESSLINGEN

- Fehlerfrei beendete Programme liefern immer den Rückgabewert `0`, ansonsten *ungleich Null*.
- Die Fehlerfreiheit kann man in Bash abfragen mittels:
  - `&&` Programm fehlerfrei beendet, und
  - `||` Programm fehlerbehaftet beendet
    - `$ grep -qi "gold" file.txt && echo 'Found Gold!'`
    - `$ gcc -Wall -O2 -o app app.c && ./app` ☺

© Copyright Material. All rights reserved.v

BASH: MIT `FIND` DATEIEN, VERZEICHNISSE UND GERÄTE FINDEN (I)

HOCHSCHULE ESSLINGEN

- Mit `find` lassen sich Dateien, Verzeichnisse, Gerätedateien finden:
  - `$ find . -maxdepth 3 -name \*.c`
  - Ablauf: Findet alle Dateien in diesem Verzeichnis (.) und bis zu drei Unterverzeichnisse tiefer mit der Dateiendung `.c` (also C-Programmdatei).
- Achtung: Die Verknüpfung mittels Pipes ist manchmal schwierig, da UNIX Dateien Leerzeichen (und Sonderzeichen) enthalten dürfen.
  - Sonderzeichen (bspw. Leerzeichen, das `?` Oder der `*`) müssen mit "`\`" escaped werden!
  - `$ find . -maxdepth 3 -name \*.c -print0 | xargs -0 grep -i 'Fix'`
  - Ablauf: Findet alle Dateien in diesem Verzeichnis (.) und bis zu drei Unterverzeichnisse tiefer mit der Dateiendung `.c` (also C-Programmdatei), welche `Fix` enthalten (Groß/Kleinschreibung ignoriert).

 © Copyright Material. All rights reserved.

BASH: MIT `FIND` DATEIEN, VERZEICHNISSE UND GERÄTE FINDEN (II)

HOCHSCHULE ESSLINGEN

- Die Syntax für `find` lautet:
  - `$ find DIR [Parameter]*` , wobei die Parameter sein können:
  - `-type X` Suche nach Typ u.a. `f` (file), `d` (directory), `l` (soft-link)
  - `-name X` Name der Datei/Verzeichnis entspricht X (einem Reg. Ausdruck)
  - `-iname X` Name der Datei/Verzeichnis entspricht X (einem Reg. Ausdruck), Groß-/Kleinschreibung wird ignoriert
  - `-ctime -1` Verändert vor weniger als 24h, bspw. kann so eine Datei dann in‘s (tägliche) Backup!
  - `-user X` Datei gehört Nutzer X (auch als numerischer Wert)
  - `-nouser` Datei gehört keinem gültigen Nutzer (nicht in `/etc/passwd`)
  - `-nogroup` Datei gehört keiner gültigen Gruppe (nicht in `/etc/group`)

 © Copyright Material. All rights reserved.

BASH: SONDERZEICHEN UND STRING-BEHANDLUNG (I)

HOCHSCHULE ESSLINGEN

- Bestimmte Sonderzeichen müssen geschützt werden, sollten diese bspw. In einem Dateinamen vorkommen:
  - `/` → Verzeichnistrenner (das wäre aber als Teile eines Dateinamens **sehr** ungewöhnlich...)
  - `?` → WildCard für ein beliebiges Zeichen
  - `*` → WildCard für eine beliebige Zeichenkette:

    `$ ls bo*t` listet Dateien `boat`, `bolt` und `boot`... Und eben `bot`.
  - `[ ]` → Ein Teil eines regulären Ausdrucks für eine Zeichenauswahl

    `$ ls bo[ao]t` listet die Dateien `boat` und `boot`... Aber nicht `bolt` oder wieder `bot`.
  - `[A-Z]` → Ein Zeichenbereich

    `$ ls bo[a-o]t` listet die Dateien `boat`, `bolt` ... Aber auch `bobt`, `boct`, `bodt`, usw.
  - …

 © Copyright Material. All rights reserved.

BASH: SONDERZEICHEN UND STRING-BEHANDLUNG (II)

HOCHSCHULE ESSLINGEN

- Bestimmte Sonderzeichen müssen geschützt werden, sollten diese bspw. In einem Dateinamen vorkommen:
  - …
  - `<`, `>` → Ein Sonderzeichen um Ein- und Ausgabe umzuleiten

    `$ program > log.txt` leitet die Std.-Ausgabe nach `log.txt`, aber nicht die Fehlerausgabe
  - `|` `||` `;` `&&` → Verknüpfung von Programmen; einzig neu „;“ als Trenner von Befehlen:

    `$ programm1 ; programm2` wie wenn die Programme auf zwei Zeilen geschrieben wären
  - `(`, `)` → Zusammenfassung von Programmen & Befehlen zu Gruppen

 © Copyright Material. All rights reserved.

BASH: SONDERZEICHEN UND STRING-BEHANDLUNG (III)

HOCHSCHULE ESSLINGEN

- Besondere Beachtung sollten Anführungszeichen finden:
  - `"x"` → Ein String (also inklusive Leerzeichen), wird von der Shell noch interpretiert, d.h. darin enthaltene `$var` und andere Sonderzeichen werden textuell ersetzt!
  - `'x'` → Ein String (also inklusive Leerzeichen), wird **nicht** interpretiert. Man kann also getrost auch Dollars übergeben: `echo 'mucho $$!'`
  - `` `x` `` → Erzeugen einer Sub-Shell, bspw. zum Erzeugen von Eingabe. Die Alternative `$(x)` ist deutlich besser lesbar (s.u.)!
- Tatsächlich haben alle ihre Berechtigung.

- **Für Profis:** Eine Subshell erlaubt das Evaluieren, bspw. einer Argumentenliste:

```
$ for i in $(cat stud_list.txt); do du -hs $i ; done
```

Das `i` nimmt die Werte an, `du` liefert Disk Usage der Verzeichnisse der Studies!

 © Copyright Material. All rights reserved.

BASH: DAS $ STEUERZEICHEN

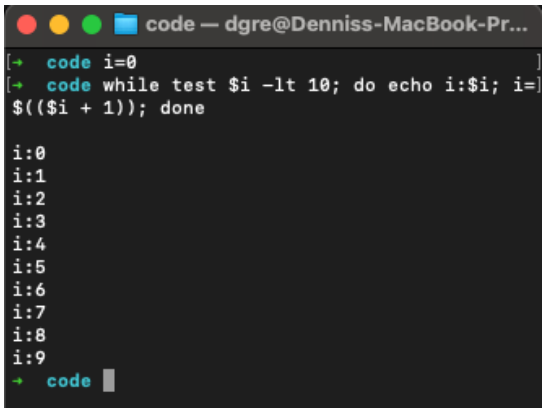
HOCHSCHULE ESSLINGEN

- Das $ ist ein besonders Sonderzeichen. Die Bash interpretiert folgendes:
  - `$0` → Name des ausgeführten Scripts, wenn keines ausgeführt: `"-bash"`
  - `$1` → Erster dem Script übergebener Parameter (`$2` ist zweiter, usw.)
  - `$*` → Alle Parameter zusammengeführt, aber durch Leerzeichen `$IFS` getrennt)
  - `$#` → Die Anzahl der übergebenen Parameter
  - `$@` → Die einzelnen Parameter
  - `$?` → Rückgabewert des zuletzt beendeten Programms
  - `$$` → Die Prozess-ID (PID) des aufrufenden Skripts / der Bash
  - `$!` → PID des letzten, im Hintergrund laufenden Prozesses:

```
$ sleep 60 &    # Schläft 60 sek., Shell kommt zurück
[1] 1234        # und liefert u.a. die PID von sleep
$ echo $!       # mit $! bekommt man die PID direkt!
```

- Mit `$(( ... ))` wird ein arithmetischer Ausdruck evaluiert, bspw.

```
$ i=0
$ while test $i -lt 10; do echo i:$i; i=$(($i + 1)); done
```



© Copyright Material. All rights reserved.

BASH: `TEST` PROGRAMM

HOCHSCHULE ESSLINGEN

- Mit Hilfe des Bash-Programms `test` können Bedingungen geprüft werden:
  - `-e file.txt` → Parameter wird als Datei interpretiert, und prüft auf Existenz!
  - `-d FILE OR DIR` → Ob dies ein Verzeichnis / Directory ist
  - `-f file.txt` → Ob dies ein File ist...
  - `-r file.txt` → Datei ist lesbar (readable), `-w` writeable, `-x` executable
  - `-s file.txt` → Datei ist größer 0 Bytes

- Strings können ebenso getestet werden:
  - `-z` → Parameter als String interpretiert, prüft Länge auf Null (**z**ero)

    ```
    $ s=""; test -z "$s" && echo "s ist 0 Zeichen lang"
    ```
  - `test s1 = s2` prüft ob String `s1` und `s2` gleich sind

- Numerische Tests:
  - `N -le M` prüft ob `N` kleiner gleich `M` ist (**l**ower or **e**qal)
  - `N -gt M` prüft ob `N` größer gleich `M` ist (**g**reater **t**han)

 © Copyright Material. All rights reserved.

BASH: TAR PROGRAMM

HOCHSCHULE ESSLINGEN

- Mit Hilfe des *Tape Archive* (kurz: `tar`) Programm lassen sich Backups aller Dateien erzeugen. Je nach Parameter und Einstellung werden die Daten direkt komprimiert (verbraucht weniger Speicherplatz):
  - `$ tar [parameter]*`
  - `-c` → Create: Erstellt neue `tar` Datei
  - `-f file.tar` → FileName: Name der Datei, die gepackt werden soll
  - `-v` → Verbose: Konsolenausgaben über aktuellen Prozesschritt
  - `-x file.tar` → Extract: Entpackt eine `tar` Datei zu einem Verzeichnis

- Beispiel:
  - ```
    $ find . -ctime -1 -print0 |
      tar -cvf tar-$(date '+%Y.%m.%d').tar --null -T -
    ```
  - Ablauf: Auflisten aller Dateinamen im aktuellen Verzeichnis (.) und erstellen eines tar backups.

 © Copyright Material. All rights reserved.
    ```

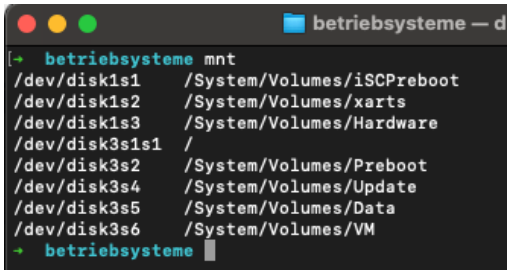
TIPPS UND TRICKS

Quelle: [Image by rawpixel.com on Freepik]

TIPPS UND TRICKS: ALIAS

HOCHSCHULE ESSLINGEN

- Mit Hilfe von `alias` lassen sich Bash Befehle umprogrammieren:
  - `$ alias l="ls -al"` → nur noch kleines `l` für Auflisten der Dateien
  - `$ alias mnt="mount |`
    `awk -F' ' '{ printf \"%s\t%s\n\",\$1,\$3; }' |`
    `egrep ^/dev/ |`
    `sort"`
    - Ablauf: Führt den neuen Befehl “mnt” ein, der alle aktuell eingebundenen Geräte (Spalte 1) auflistet, nach Geräte-Namen zusammen mit dem „mount-point“ (Spalte 2)



- **Hinweis**: Um vorgenommene Einstellungen rückgängig zu machen, hilft das aus- und wieder einloggen.

© Copyright Material. All rights reserved.

TIPPS UND TRICKS: `.BASHRC` / `.ZSHRC`

HOCHSCHULE ESSLINGEN

- Beim Einloggen des Benutzers wird standardmäßig die Datei `~/.bashrc` gelesen.
  - Hier kann man Änderungen vornehmen um eigene Kürzel, Befehle oder Programme zu laden
  - **Hinweis**: Da macOS die z-Shell benutzt können Änderungen in der `~/.zshrc` vorgenommen werden.

 © Copyright Material. All rights reserved.

TIPPS UND TRICKS: `TOP` PROGRAMM

- Das `top` Programm bietet eine dynamische Echtzeitansicht auf das laufende Linux-System. Es kann Systemzusammenfassungen, sowie eine Liste der Prozesse und verwendeter Ressourcen anzeigen.

dgre — top — top — top — 160×55

```
Processes: 632 total, 2 running, 630 sleeping, 3179 threads                                                                    21:08:16
Load Avg: 1.64, 1.38, 1.31  CPU usage: 0.94% user, 1.37% sys, 97.68% idle  SharedLibs: 778M resident, 150M data, 56M linkedit.
MemRegions: 387188 total, 5623M resident, 417M private, 3337M shared. PhysMem: 16G used (2135M wired, 1483M compressor), 1110M unused.
VM: 273T vsize, 4921M framework vsize, 0(0) swapins, 0(0) swapouts. Networks: packets: 323908/278M in, 150421/38M out.
Disks: 476030/13G read, 375962/11G written.

PID    COMMAND      %CPU TIME     #TH    #WQ  #PORT MEM    PURG  CMPRS  PGRP  PPID STATE    BOOSTS         %CPU_ME %CPU_OTHRS UID  FAULTS   COW   MSGSENT
85822  top          5.0  00:02.55 1/1    0    29    20M    0B    0B     85822 670  running  *0[1]          0.00000 0.00000    0    22560+   81    1403956+
150    WindowServer 4.3  30:36.96 19     5    3109- 760M-  245M- 41M    150   1    sleeping *0[1]          0.04181 0.53504    88   1492053+ 8570  13641087+
0      kernel_task  3.7  11:28.10 604/11 0    0     3424K  0B    0B     0     0    running   0[0]          0.00000 0.00000    0    9077     0     17314847+
774    Podman Deskt 2.4  05:42.54 43     10   428   164M   64K   79M    774   1    sleeping *0[1]          0.08408 0.00000    501  2657906+ 8753  1168656+
594    Google Chrom 1.9  01:37.10 21     1    171   75M    0B    28M    460   460  sleeping *0[5]          0.00000 0.00000    501  19689    749   1857578+
2638   zoom.us      1.6  03:47.95 40     3    647   149M   0B    70M    2638  1    sleeping  0[1289]       0.00000 0.00000    501  56308+   1016  1822606+
572    Google Chrom 1.2  08:40.37 14     1    316   379M   248M  34M    460   460  sleeping *1[2]          0.00000 0.00000    501  750635   876   8007341+
2697   ZoomClips    0.8  01:25.12 23     3    378   48M    0B    25M    2697  2638 sleeping *0[101]        0.00000 0.00000    501  19944    491   517209+
508    Calendar     0.7  00:10.58 5      3    417   122M   0B    26M    508   1    sleeping  0[319]        0.05179 0.00000    501  32418    325   96672+
405    cfprefsd     0.5  00:07.04 3      2    607   3953K  112K+ 640K   405   1    sleeping *0[5438+]      0.00000 0.04630    501  7405+    83    17858+
513    Terminal     0.5  00:15.68 10     3    394-  248M-  164M- 12M    513   1    sleeping *0[351+]       0.06496 0.02527    501  41471+   257   270976+
49377  Google Chrom 0.5  00:52.28 25     1    253   160M   0B    23M    460   460  sleeping *0[5]          0.00000 0.00000    501  64342    760   138318
```

© Copyright Material. All rights reserved.

BETRIEBSSYSTEME SERVER: BS_LAB

HOCHSCHULE ESSLINGEN

- Für jeden Teilnehmer dieses Kurses existiert **ein** Account auf dem Betriebssysteme-Server bslab-it:
  - Login: `$ ssh rakeller@bslab-it.hs-esslingen.de`
  - Jeder Nutzer hat:
    - eine eindeutige User-ID (UID), diese steht in `/etc/passwd`
    - Gehört min. zu einer Benutzer-gruppe (z.B.: zu `staff`)
  - Jeder Nutzer darf:
    - (fast) beliebig viele Prozesse starten (Hinweis: Ein Programm besteht aus mind. einem Prozess)
    - **Manche** Nutzer dürfen SuperUser root werden, mit su - oder mit sudo.
      - **Vorsicht**: Dieser Nutzer darf alles, wirklich **ALLES**!!!

 © Copywright Material. All rights reserved.

ZUSAMMENFASSUNG

- Kennenlernen der Bash und des Funktionsumfangs
- Vorstellung der wichtigsten Bash Befehle
- Tipps und Trick im Umgang mit der Bash

 © Copyright Material. All rights reserved.